

Closing the Definition Dependency Gap: Pre-Embedding Definition Injection for Legal Contract Retrieval

Kamal Goyal

Kaytech Software Canada Inc.

Abstract

While building a Canadian legal research platform over the CUAD corpus and CanLII case law, I kept running into the same failure: queries about specific contract obligations returned the wrong clauses. Digging into why, I found the cause was straightforward but unfixed -- every RAG pipeline I tested embedded clauses without the definitions that give those clauses their legal meaning. A clause on page 47 referencing a "Material Adverse Effect" carries no semantic signal about what that term means, because the definition sitting on page 3 never made it into the embedding. I call this the Definition Dependency Gap (DDG). This paper describes DAPEI -- Definition-Aware Pre-Embedding Injection -- a preprocessing pipeline I built to fix it. DAPEI works in three steps: it extracts defined terms from a contract using a cascaded regex, structural, and LLM-based approach; it resolves nested definitions by building a dependency graph and topologically sorting it so that a term whose definition references another defined term gets the full resolved meaning; and it injects those resolved definitions directly into each chunk before the chunk is embedded, so the vector actually represents what the clause means -- not just what it says. I also introduce DefinitionBench, a benchmark built on CUAD contracts that separates queries into those that do and do not require definition resolution, making it possible to measure exactly how much the DDG costs in retrieval quality. On definition-dependent queries, DAPEI improves Precision@5 and MRR substantially over naive chunking and post-retrieval injection baselines. The code and benchmark are released openly. This is a practical fix to a problem that is causing real retrieval failures in production legal AI systems today.

1 Introduction

I started building a legal research platform for Canadian law firms in 2024. The first version used a standard RAG stack -- pdfplumber to extract text, recursive character splitting at 512 tokens, OpenAI embeddings, ChromaDB. It worked fine on general questions. But when lawyers asked about specific contract obligations -- "what are the indemnification triggers under this agreement?" or "when does the non-compete apply?" -- the retrieved chunks were consistently wrong. Not slightly wrong. The system would return clauses from completely different sections that happened to share surface-level vocabulary with the query.

I spent several weeks debugging this before the pattern became clear. The failing queries all involved defined terms. A clause reading "the Indemnified Party shall give written notice within 30 days of any Triggering Event" sits on page 31 of a contract. Its embedding knows nothing about "Indemnified Party" (defined on page 4) or "Triggering Event" (defined on page 6, cross-referencing Schedule B). The embedding model -- trained on general English -- treats these as ordinary noun phrases. The resulting vector matches queries about notification procedures, not indemnification. Retrieval fails before the

language model even sees the chunk.

This is not a language model problem. It is a pipeline problem. No amount of prompt engineering or model improvement fixes retrieval failures caused by embeddings that were computed without the information they needed. I call this the Definition Dependency Gap (DDG): the semantic distance between a chunk's embedding and its true contractual meaning, caused by the absence of defined-term context at embedding time.

A 2025 Stanford study found Lexis+ AI and Westlaw hallucinate on 17% and 33% of legal queries respectively. The study measured output-level failures without tracing them to pipeline stages. Based on my experience building a legal RAG system from scratch, I believe the DDG is a major structural contributor to those numbers -- and it is fixable at the pipeline level without changing the language model at all.

This paper describes the fix I built. The contributions are:

- (1) A formal definition of the DDG as a distinct failure mode in legal RAG.
- (2) DAPEI -- a preprocessing pipeline that closes the DDG through pre-embedding definition injection with nested term resolution.
- (3) DefinitionBench -- a benchmark stratified by definition dependency so the DDG's retrieval cost can be measured directly.
- (4) A full open-source implementation reproducible from a single command.

2 Related Work

Legal NLP benchmarks. When I went looking for a benchmark that would let me measure the DDG directly, I could not find one. CUAD (Hendrycks et al., 2021) has 510 annotated contracts with 41 clause categories -- useful as a corpus but not designed to isolate definition-dependent retrieval. ContractNLI (Koreeda & Manning, 2021) tests entailment over NDAs. LegalBench (Guha et al., 2023) covers 162 legal reasoning tasks but explicitly scopes out cross-document and cross-reference reasoning. ACORD (2025) is the closest -- expert-annotated clause retrieval -- and its authors acknowledge that cross-references add complexity, then leave that complexity out of scope. The absence of a definition-stratified benchmark is itself part of the problem: if you cannot measure the DDG, you cannot tell whether you have fixed it.

RAG chunking strategies. Recursive character splitting is the default approach in every major framework. Summary-Augmented Chunking (SAC; arxiv 2510.06999) prepends a short document summary to each chunk -- this helps with document-level context but does nothing for clause-level definition semantics. Parent-child chunking returns larger chunks on retrieval, which helps with adjacent context but still does not propagate definitions from page 3 to page 47. Anthropic's Contextual Retrieval (2024) uses an LLM to write a situating sentence for each chunk; I tested this and found it helps on general queries but the LLM-generated context rarely includes the specific defined terms a clause depends on, because the LLM summarises what a chunk says rather than what its vocabulary means in this contract.

Graph-based legal RAG. ComplianceNLP (2026) builds a regulatory knowledge graph with cross-reference edges and achieves strong results on regulatory norm retrieval. This is the most technically sophisticated related work, but it targets regulatory provisions (SEC, MiFID II, Basel III) not commercial contracts, and it does not address pre-embedding injection. A practitioner prototype published on Medium (Enterprise RAG, 2024) builds a separate definitions graph and injects definitions after retrieval. I tried this approach first. It helps the language model read correctly but does not fix the retrieval step -- you still retrieve the wrong chunks because their embeddings were computed without definition context.

The key difference in DAPEI is where injection happens: before embedding, not after retrieval. This is the distinction that matters for retrieval quality.

3 Method

3.1 Problem Formulation

Let D be a legal contract consisting of sections $\{s_1, s_2, \dots, s_n\}$. A standard RAG pipeline decomposes D into chunks $\{c_1, c_2, \dots, c_k\}$, embeds each chunk independently to produce vectors $\{v_1, \dots, v_k\}$, and at query time retrieves the top- K chunks by cosine similarity to a query embedding v_q .

Legal contracts define a vocabulary $T = \{(t_i, d_i)\}$ where each term t_i is assigned a definition d_i , typically in a dedicated Definitions section. When a chunk c_j contains a reference to term t_i but is embedded without d_i , the resulting vector v_j is semantically incomplete: it represents the surface form of t_i rather than its contractual meaning. We define this failure mode as the Definition Dependency Gap (DDG).

The DDG is particularly acute when d_i itself references other defined terms (nested definitions), creating chains of semantic dependency that naive chunking cannot capture. We define a definition-dependent query as a query q whose correct answer requires knowing d_i for at least one t_i in T that appears in the relevant chunk. Our goal is to produce chunk embeddings that carry the semantic content of all relevant definitions, enabling accurate retrieval on definition-dependent queries.

3.2 Architecture Overview

DAPEI inserts three processing modules between document parsing and embedding. The pipeline proceeds as follows: (1) the Definition Extractor processes the parsed contract to recover the vocabulary T ; (2) the Dependency Resolver constructs a directed term graph and produces fully resolved definitions accounting for nested term references; (3) the Structure-Aware Chunker segments the document respecting section boundaries; (4) the Chunk Enricher prepends a definitions block to each chunk prior to embedding; and (5) the embedding model computes vectors on enriched text. The vector store retains original chunk text for LLM retrieval, using enriched text only for indexing.

At query time, queries containing defined terms are also enriched with a compact definitions block before embedding, ensuring alignment between query and chunk vector spaces.

3.3 Definition Extractor

My first version of the extractor was a single regex: quoted-term followed by 'means'. It worked on about half the contracts I tested. The failures came from three sources: contracts that used 'shall mean' instead of 'means'; contracts that defined terms inline with parentheticals like (the "Agreement") rather than in a Definitions section; and contracts that used unquoted capitalised terms at the start of a line. I kept adding patterns until I had six, covering the syntactic forms I encountered across 200+ EDGAR exhibits. That got Layer 1 coverage to around 61% of contracts.

Layer 1 -- Regex. Six patterns in order of specificity: (i) "Term" means [definition]; (ii) single-quote variant; (iii) bare capitalised term at line start followed by means; (iv) inline parenthetical: [text] (the "Term"); (v) scoped preamble: As used in this Agreement, "Term" means [definition]; (vi) qualified means with embedded qualifier clause. Deduplication by lowercased term key, higher-specificity patterns win on collision.

Layer 2 -- Structural. When a section heading contains 'Definition', 'Interpretation', or 'Meanings', I switch to paragraph-level scanning. Each paragraph gets checked for a line-initial capitalised term followed by a definitional verb. This catches the non-standard formatting that escapes the regex patterns. Adding this layer raised coverage to 78%.

Layer 3 -- LLM fallback. For confirmed Definitions sections that still yield fewer than three extractions after Layers 1 and 2, I call GPT-4o-mini with a structured extraction prompt and parse the JSON response. I set the threshold at three rather than zero because short Definitions sections are often boilerplate (one or two terms like 'Agreement' and 'Parties') and do not justify an API call. This layer runs only on confirmed Definitions sections, keeping the per-contract LLM cost under \$0.002 on average. Combined coverage reaches approximately 91%.

3.4 Dependency Resolver

I initially injected raw definitions without resolving nesting. Then I hit a contract where 'Material Adverse Effect' was defined as any event materially affecting the 'Business', 'Business' was defined in terms of the 'Company' and its 'Subsidiaries', and 'Subsidiaries' had its own definition. A query about Material Adverse Effect needed all three definitions to be retrievable, not just the top-level one. Running the extractor on a sample of 50 contracts, I found that 34% of extracted terms have at least one dependency (depth ≥ 1) and 11% have depth ≥ 2 . This is common enough that flat injection without resolution leaves a meaningful fraction of queries underserved.

The resolver builds a directed graph $G = (V, E)$ where each node is a defined term and each edge ($t_i \rightarrow t_j$) means t_i 's definition text contains a reference to t_j . Edges are detected with word-boundary regex that handles plurals and possessives, since contracts routinely use 'Subsidiaries' in a clause where the defined term is 'Subsidiary'.

Cycles are rare but real -- I encountered two mutual cross-references in the CUAD corpus. Detection is via DFS; cycles are broken by dropping the lowest-alphabetical edge and processing affected terms with

raw definitions.

Topological sort (Kahn's algorithm) ensures dependency-free terms are processed first. Each term's resolved definition is its raw definition plus a bracketed glossary of transitive dependencies, capped at depth 3 and 200 characters per dependency entry. The cap prevents token explosion on deeply nested agreements while preserving the most important context.

3.5 Chunk Enricher

For each chunk, the enricher scans for defined terms using word-boundary regex and ranks them by frequency of occurrence within the chunk -- terms used more often in a clause are more likely to be central to its meaning. It then builds a definitions block within a 256-token budget and prepends it to the chunk text before the embedding is computed.

The token budget decision required some tuning. My first attempt used no budget cap. On contracts with 40+ defined terms, the enrichment block overwhelmed the chunk text and retrieval got worse -- the embedding was representing the glossary, not the clause. After testing several thresholds on a held-out set of 30 contracts, 256 tokens for enrichment against a 512-token chunk gave the best retrieval performance. When a resolved definition exceeds the remaining budget, the raw (un-nested) definition is used instead; if even that is too long, the term is omitted from the block.

The most important implementation detail: the enriched text is used only to compute the embedding vector. The vector store keeps the original chunk text separately and returns that to the language model at query time. This means the LLM reads the actual contract clause, not a clause prefixed with a definitions block -- while the embedding accurately represents what the clause means in context of the contract's own vocabulary.

3.6 Cross-Reference Resolution

Legal contracts frequently defer meaning to other sections via explicit cross-references (e.g., "as defined in Section 3.1(b)(ii)"). We detect these using a section-reference pattern and resolve them by fetching the referenced section text and appending it as an inline annotation. Cross-reference resolution is applied as a fourth enrichment pass after definition injection, operating on the already-enriched chunk text.

3.7 Query-Time Enrichment

To maintain alignment between query and chunk vector spaces, queries are also enriched at retrieval time. For a query q , we detect defined terms in q using the same term-matching procedure and prepend a compact definitions block (budget 128 tokens, raw definitions only) before computing the query embedding. This ensures that a query containing a defined term is embedded with partial knowledge of that term's contractual meaning.

4 DefinitionBench

The absence of a benchmark that directly measures definition-dependent retrieval is part of why this

problem went unfixed. If your evaluation set treats all queries the same, a system that fails only on defined-term queries will look nearly as good as one that handles them correctly. DefinitionBench is designed to make that failure visible.

Construction. I use the CUAD corpus (511 commercial contracts) as the document collection because it is publicly available, covers a wide range of commercial agreement types, and comes with existing annotations. For each contract, DAPEI's Definition Extractor is run to recover the defined-term vocabulary. QA pairs are then built from the CUAD annotations and split into two types.

Type A -- Definition-independent: the correct clause can be found without knowing any defined term. These are the control group. All methods should perform similarly here; large differences would indicate something else is going wrong.

Type B -- Definition-dependent: the correct clause contains at least one defined term from that contract's vocabulary, and the question is about a clause category where defined-term meaning is typically decisive (indemnification, termination, material adverse effect, change of control, intellectual property, confidentiality). This is the test group. A method that does not address the DDG should underperform here relative to Type A.

I deliberately kept the stratification criterion objective and reproducible: no manual labelling beyond what CUAD already provides. The Type B classifier is two conditions checked programmatically. Full benchmark statistics and retrieval results across both splits are reported in the complete paper.

5 Preliminary Results

Full benchmark evaluation is ongoing. I report here the observations from an initial run across 50 CUAD contracts (~450 chunks, 180 QA pairs, 72 Type B).

Definition extraction coverage. Layer 1 regex alone recovered at least one defined term in 61% of contracts. Adding Layer 2 structural extraction raised this to 78%. The LLM fallback (Layer 3) brought coverage to approximately 91% of contracts that had identifiable Definitions sections. The remaining 9% were contracts where definitions were scattered inline throughout the body with no dedicated section -- these are harder and represent a known limitation of the current approach.

Nesting depth distribution. Of all extracted terms, 34% had at least one dependency (depth ≥ 1) and 11% had depth ≥ 2 . The deepest chain I encountered in the CUAD corpus was depth 4 -- a term whose definition referenced a term whose definition referenced a term whose definition referenced a schedule. The 3-hop cap in the resolver handles this correctly by truncating at the schedule reference rather than trying to parse the schedule itself.

Retrieval comparison (preliminary). On Type A queries (definition-independent), DAPEI and naive chunking performed comparably -- as expected, since definitions do not affect these queries. On Type B queries, DAPEI showed clear improvement in Precision@5 and MRR over naive chunking and

post-retrieval injection. Full metrics across all baselines (naive chunking, Summary-Augmented Chunking, Parent-Child, and post-retrieval injection) will be reported in the complete paper alongside DefinitionBench statistics.

6 Conclusion

The Definition Dependency Gap is not a subtle or edge-case problem. It affects every contract that has a Definitions section -- which is most of them -- and it causes retrieval failures on exactly the queries lawyers care most about: indemnification triggers, termination conditions, change of control provisions. These are not general vocabulary questions. They are questions about what specific words mean inside a specific document. A retrieval system that cannot answer them reliably is not useful for legal work.

I built DAPEI because I needed it to work on a real platform serving real users. The fix turned out to be simpler than I expected: extract the definitions, resolve the nesting, inject them before embedding. The hard part was realising that post-retrieval injection -- which is what the existing prototype work does -- does not actually fix the problem because it does not fix the retrieval step.

There is still meaningful work left. The 9% of contracts with no dedicated Definitions section need a different approach -- probably clause-level entity recognition rather than section-level extraction. Multi-document definition resolution (master agreements with schedules and exhibits) is unsolved. And the Canadian legal market, where I am building this, has the added complexity of bilingual documents and Quebec civil law terminology that behaves quite differently from common law contract language.

The full implementation, benchmark, and evaluation code are released openly. If you are building a legal RAG system and hitting the same retrieval failures I was, this should give you a working starting point.